

Pandas – real world?

Pivot, merge, concat, melt

My example

- I analyze data from the HBCD (Healthy Brain and Child Development) study. You can learn more about the released data from <https://www.nbdc-datahub.org/hbcd-release-1-0>
- Specifically, I analyze wearable sensor data that recorded infants' leg movements for 72 hours. Data are prepared following motion-BIDS (<https://bids-specification.readthedocs.io/en/stable/modality-specific-files/motion.html>)

My example

- Different types of data are saved in different folders/formats

Processed data

File 1: Values of movement kinematics
(.JSON)

File 2: Physical activity intensity level
classification (.JSON)

Metadata

File 3: Timestamps of when did sensors
start recording + age of an infant (.TSV)

File 4: Recording related configurations
(.JSON)

My example

- File 1: Movement kinematics

See how this a JSON file is basically a Python dictionary?
(We introduced this in week 1)

So the outermost level keys are:
“Participant_id”, “Session_id”, etc.

Do you recognize a nested dictionary?
To access the value of “Left_average_acc_SampEn”, we
previously introduced these lines of code:

```
# `file_path` is the full path to the .json file
# How to open a .json file -> we saw this in week 1.
with open(file_path, 'rb') as f:
    data = json.load(f)
```

```
In [10]: data['entropies']['Left_average_acc_SampEn']
Out[10]: 1.2872497310617599
```

```
{
  "Participant_id": "sub-1526180984",
  "Session_id": "ses-V02",
  "Left_positive_threshold": 1.0675900497758095,
  "Right_positive_threshold": 1.0796716514895888,
  "Left_negative_threshold": -1.0537033458130614,
  "Right_negative_threshold": -1.0058029372497637,
  "Left_total_count": 22846,
  "Right_total_count": 27869,
  "Left_movement_rate": 831.2203877686325,
  "Right_movement_rate": 949.7629578019449,
  "Left_sleep_time": 44.0,
  "Right_sleep_time": 43.833333333333336,
  "Left_average_acc_median": 1.2346422081357336,
  "Left_average_acc_igr": 0.9569404705506,
  "Left_peak_acc_median": 2.5593015346128296,
  "Left_peak_acc_igr": 2.29308108702331,
  "Left_movement_duration_median": 0.32,
  "Left_movement_duration_igr": 0.16000000000000003,
  "Right_average_acc_median": 1.3014025013830044,
  "Right_average_acc_igr": 1.1408569271133655,
  "Right_peak_acc_median": 2.703437817200582,
  "Right_peak_acc_igr": 2.780495445775893,
  "Right_movement_duration_median": 0.28,
  "Right_movement_duration_igr": 0.16000000000000003,
  "entropies": {
    "Left_average_acc_SampEn": 1.2872497310617599,
    "Left_average_acc_FuzzEn": 0.8021761279058033,
    "Left_peak_acc_SampEn": 1.1040278704986637,
    "Left_peak_acc_FuzzEn": 1.4065905636825677,
    "Right_average_acc_SampEn": 1.1547017425931392,
    "Right_average_acc_FuzzEn": 0.88025572204146,
    "Right_peak_acc_SampEn": 0.9647095842289395,
    "Right_peak_acc_FuzzEn": 1.467185983821936
  }
}
```

A dictionary to a DataFrame

- A simple conversion does not work easily...

```
df1 = pd.DataFrame(data)
```

Index	t_average_acc	_peak_acc_me	ft_peak_acc_i	ement_duration	ovement_dural	average_acc_r	it_average_acc	_peak_acc_m	jht_peak_acc_	rement_duratic	ovement_dura	entropies
Left_average_acc_SampEn	0.95694	2.5593	2.29308	0.32	0.16	1.3014	1.14086	2.70344	2.7805	0.28	0.16	1.28725
Left_average_acc_FuzzEn	0.95694	2.5593	2.29308	0.32	0.16	1.3014	1.14086	2.70344	2.7805	0.28	0.16	0.802176
Left_peak_acc_SampEn	0.95694	2.5593	2.29308	0.32	0.16	1.3014	1.14086	2.70344	2.7805	0.28	0.16	1.10403
Left_peak_acc_FuzzEn	0.95694	2.5593	2.29308	0.32	0.16	1.3014	1.14086	2.70344	2.7805	0.28	0.16	1.40659
Right_average_acc_SampEn	0.95694	2.5593	2.29308	0.32	0.16	1.3014	1.14086	2.70344	2.7805	0.28	0.16	1.1547
Right_average_acc_FuzzEn	0.95694	2.5593	2.29308	0.32	0.16	1.3014	1.14086	2.70344	2.7805	0.28	0.16	0.880256
Right_peak_Acc_SampEn	0.95694	2.5593	2.29308	0.32	0.16	1.3014	1.14086	2.70344	2.7805	0.28	0.16	0.96471
Right_peak_Acc_FuzzEn	0.95694	2.5593	2.29308	0.32	0.16	1.3014	1.14086	2.70344	2.7805	0.28	0.16	1.46719

Nested dictionary keys
are read as the index of
this DataFrame

I want more like this!

Index	Participant_id	Session_id	_positive_thres	_positive_thre	_negative_thres	_negative_thre	_eft_total_coun	ight_total_coun	t_movement_r	it_movement_	_eft_sleep_time	ight_sleep_tim	verage_acc_
0	sub-1526180984	ses-V02	1.06759	1.07967	-1.0537	-1.0058	22846	27869	831.22	949.763	44	43.8333	1.23464

A dictionary to a DataFrame

- Step 1: Make the index into a column.

```
# Use .reset_index() to get the nested dictionary's keys  
# We now know what `inplace=True` does, right?  
df1.reset_index(inplace=True)
```

Use this command to access the
'Index' of the DataFrame

`df1.index`

Use this command to access the
column named 'index' of the
DataFrame

`df1['index']`

Index	Left_average_acc	Left_peak_acc	Right_average_acc	Right_peak_acc	element_duration
Left_average_acc_SampEn	0.95694	2.5593	2.29308	0.32	
Left_average_acc_FuzzEn	0.95694	2.5593	2.29308	0.32	
Left_peak_acc_SampEn	0.95694	2.5593	2.29308	0.32	
Left_peak_acc_FuzzEn	0.95694	2.5593	2.29308	0.32	
Right_average_acc_SampEn	0.95694	2.5593	2.29308	0.32	
Right_average_acc_FuzzEn	0.95694	2.5593	2.29308	0.32	
Right_peak_Acc_SampEn	0.95694	2.5593	2.29308	0.32	
Right_peak_Acc_FuzzEn	0.95694	2.5593	2.29308	0.32	

Index	index	Participant_id	Session_id	_positive_thres	_positive_thre:
0	Left_average_acc_SampEn	sub-1526180984	ses-V02	1.06759	1.07967
1	Left_average_acc_FuzzEn	sub-1526180984	ses-V02	1.06759	1.07967
2	Left_peak_acc_SampEn	sub-1526180984	ses-V02	1.06759	1.07967
3	Left_peak_acc_FuzzEn	sub-1526180984	ses-V02	1.06759	1.07967
4	Right_average_acc_SampEn	sub-1526180984	ses-V02	1.06759	1.07967
5	Right_average_acc_FuzzEn	sub-1526180984	ses-V02	1.06759	1.07967
6	Right_peak_Acc_SampEn	sub-1526180984	ses-V02	1.06759	1.07967
7	Right_peak_Acc_FuzzEn	sub-1526180984	ses-V02	1.06759	1.07967

A dictionary to a DataFrame

- Step 2: Prepare two DataFrames.

```
# Make a DataFrame with the following three columns
df2 = df1[['Participant_id', 'entropies', 'index']]
# Drop the two columns: 'entropies', 'index'
df1.drop(columns=['entropies', 'index'], inplace=True)
```

df1: 'entropies' and 'index' columns dropped

	Index	Participant_id	Session_id	_positive_thres
	0	sub-1526180984	ses-V02	1.06759
	1	sub-1526180984	ses-V02	1.06759
	2	sub-1526180984	ses-V02	1.06759
	3	sub-1526180984	ses-V02	1.06759
	4	sub-1526180984	ses-V02	1.06759
	5	sub-1526180984	ses-V02	1.06759
	6	sub-1526180984	ses-V02	1.06759
	7	sub-1526180984	ses-V02	1.06759

df2: 'Participant_id', 'entropies' & 'index'

	Index	Participant_id	entropies	index
	0	sub-1526180984	1.28725	Left_average_acc_SampEn
	1	sub-1526180984	0.802176	Left_average_acc_FuzzEn
	2	sub-1526180984	1.10403	Left_peak_acc_SampEn
	3	sub-1526180984	1.40659	Left_peak_acc_FuzzEn
	4	sub-1526180984	1.1547	Right_average_acc_SampEn
	5	sub-1526180984	0.880256	Right_average_acc_FuzzEn
	6	sub-1526180984	0.96471	Right_peak_Acc_SampEn
	7	sub-1526180984	1.46719	Right_peak_Acc_FuzzEn

A dictionary to a DataFrame

- Step 3: Make df2 a wide format.

Index	Participant_id	entropies	index
0	sub-1526180984	1.28725	Left_average_acc_SampEn
1	sub-1526180984	0.802176	Left_average_acc_FuzzEn
2	sub-1526180984	1.10403	Left_peak_acc_SampEn
3	sub-1526180984	1.40659	Left_peak_acc_FuzzEn
4	sub-1526180984	1.1547	Right_average_acc_SampEn
5	sub-1526180984	0.880256	Right_average_acc_FuzzEn
6	sub-1526180984	0.96471	Right_peak_Acc_SampEn
7	sub-1526180984	1.46719	Right_peak_Acc_FuzzEn

```
# Make it a wide format: use `.pivot()` or `.pivot_table()`  
df2_wide = df2.pivot(index='Participant_id',  
                      columns='index',  
                      values='entropies')  
df2_wide_v2 = df2.pivot_table(index='Participant_id',  
                               columns='index',  
                               values='entropies')
```

Column(s) to use to
make a new
DataFrame's columns

Column(s) to use for
populating new
DataFrame's values

Participant_id	Left_average_acc_FuzzEn	Left_average_acc_SampEn	Left_peak_acc_FuzzEn	Left_peak_acc_SampEn	Right_average_acc_FuzzEn
sub-1526180984	0.802176	1.28725	1.40659	1.10403	0.880256

A dictionary to a DataFrame

- Step 3: Make df2 a wide format.

	Index	Participant_id	entropies	index
0		sub-1526180984	1.28725	Left_average_acc_SampEn
1		sub-1526180984	0.802176	Left_average_acc_FuzzEn
2		sub-1526180984	1.10403	Left_peak_acc_SampEn
3		sub-1526180984	1.40659	Left_peak_acc_FuzzEn
4		sub-1526180984	1.1547	Right_average_acc_SampEn
5		sub-1526180984	0.880256	Right_average_acc_FuzzEn
6		sub-1526180984	0.96471	Right_peak_Acc_SampEn
7		sub-1526180984	1.46719	Right_peak_Acc_FuzzEn

```
# Make it a wide format: use `.pivot()` or `.pivot_table()`  
df2_wide = df2.pivot(index='Participant_id',  
                     columns='index',  
                     values='entropies')  
df2_wide_v2 = df2.pivot_table(index='Participant_id',  
                              columns='index',  
                              values='entropies')
```

Column to use to make
new DataFrame's index

Participant_id	Left_average_acc_FuzzEn	Left_average_acc_SampEn	Left_peak_acc_FuzzEn	Left_peak_acc_SampEn	Right_average_acc_FuzzEn
sub-1526180984	0.802176	1.28725	1.40659	1.10403	0.880256

A dictionary to a DataFrame

- Step 3: Make df2 a wide format.

	Index	Participant_id	entropies	index
	0	sub-1526180984	1.28725	Left_average_acc_SampEn
	1	sub-1526180984	0.802176	Left_average_acc_FuzzEn
	2	sub-1526180984	1.10403	Left_peak_acc_SampEn
	3	sub-1526180984	1.40659	Left_peak_acc_FuzzEn
	4	sub-1526180984	1.1547	Right_average_acc_SampEn
	5	sub-1526180984	0.880256	Right_average_acc_FuzzEn
	6	sub-1526180984	0.96471	Right_peak_Acc_SampEn
	7	sub-1526180984	1.46719	Right_peak_Acc_FuzzEn

Existing index

```
# Make it a wide format: use `.pivot()` or `.pivot_table()`  
df2_wide = df2.pivot(index='Participant_id',  
                      columns='index',  
                      values='entropies')  
df2_wide_v2 = df2.pivot_table(index='Participant_id',  
                               columns='index',  
                               values='entropies')
```

If `index` is not provided,
the existing index of `df2`
will be used

	Index	average_acc_FuzzEn	average_acc_SampEn
	0	nan	1.28725
	1	0.802176	nan
	2	nan	nan
	3	nan	nan
	4	nan	nan
	5	nan	nan
	6	nan	nan
	7	nan	nan

A dictionary to a DataFrame

- Step 4: Merge the two.

df2_wide

Participant_id	average_acc_F	average_acc_S	_peak_acc_Fu
sub-1526180984	0.802176	1.28725	1.40659



df1

	Index	Participant_id	Session_id	_positive_thres
	0	sub-1526180984	ses-V02	1.06759
	1	sub-1526180984	ses-V02	1.06759
	2	sub-1526180984	ses-V02	1.06759
	3	sub-1526180984	ses-V02	1.06759
	4	sub-1526180984	ses-V02	1.06759
	5	sub-1526180984	ses-V02	1.06759
	6	sub-1526180984	ses-V02	1.06759
	7	sub-1526180984	ses-V02	1.06759

To merge the two, we need to make things *consistent*:

- 1) Set 'Participant_id' as the index of df1
- 2) Drop the duplicates of df1 to make df1 a single row DataFrame.

A dictionary to a DataFrame

- Step 4a: Set 'Participant_id' as the index of df1.

```
df1.set_index('Participant_id', inplace=True)
```

	Index	Participant_id	Session_id	_positive_thres
	0	sub-1526180984	ses-V02	1.06759
	1	sub-1526180984	ses-V02	1.06759
	2	sub-1526180984	ses-V02	1.06759
	3	sub-1526180984	ses-V02	1.06759
	4	sub-1526180984	ses-V02	1.06759
	5	sub-1526180984	ses-V02	1.06759
	6	sub-1526180984	ses-V02	1.06759
	7	sub-1526180984	ses-V02	1.06759

[illegible]

A dictionary to a DataFrame

- Step 4b: Drop the duplicates.

```
df1.drop_duplicates(inplace=True)
```

Participant_id	Session_id	_positive_thres
sub-1526180984	ses-V02	1.06759
sub-1526180984	ses-V02	1.06759
sub-1526180984	ses-V02	1.06759
sub-1526180984	ses-V02	1.06759
sub-1526180984	ses-V02	1.06759
sub-1526180984	ses-V02	1.06759
sub-1526180984	ses-V02	1.06759
sub-1526180984	ses-V02	1.06759



Participant_id	Session_id	_positive_thres	_positive_thres
sub-1526180984	ses-V02	1.06759	1.07967

A dictionary to a DataFrame

This indicates that df1 is the “left” DataFrame.
If the command was `df2.merge(...)` , then
df1 is the “right” DataFrame

- Step 4c: Merge the two transformed DataFrames.

```
merged = df1.merge(df2_wide, left_index=True, right_index=True)
```

df1

Participant_id	Session_id	_positive_thres	_positive_thres
sub-1526180984	ses-V02	1.06759	1.07967

This is index!

df2_wide

Participant_id	average_acc_F	average_acc_S	_peak_acc_Fu	_peak_acc_Sar	average_acc_F
sub-1526180984	0.802176	1.28725	1.40659	1.10403	0.880256

This is index!

merged

Participant_id	Session_id	_positive_thres	_positive_thres	_negative_thres	_negative_thres	left_total_coun	right_total_coun
sub-1526180984	ses-V02	1.06759	1.07967	-1.0537	-1.0058	22846	27869

A dictionary to a DataFrame

- Step 4c: Merge the two transformed DataFrames.

```
merged = df1.merge(df2_wide, left_index=True, right_index=True)
```

df1

Participant_id	Session_id	_positive_thres	_positive_thre:
sub-1526180984	ses-V02	1.06759	1.07967

df2_wide

Participant_id	average_acc_F	verage_acc_S	_peak_acc_Fu:	_peak_acc_Sar	average_acc_F
sub-1526180984	0.802176	1.28725	1.40659	1.10403	0.880256

Merge happens based on these two indices.
As the two indices are the same in value, you can
make one row DataFrame, merged

merged

Participant_id	Session_id	_positive_thres	_positive_thre:	_negative_thres	_negative_thre	left_total_coun	ight_total_coun
sub-1526180984	ses-V02	1.06759	1.07967	-1.0537	-1.0058	22846	27869

A dictionary to a DataFrame

- Step 4c can be done using different arguments: *left_on* & *right_on*

```
# What if 'index' all removed?  
df1.reset_index(inplace=True)  
df2_wide.reset_index(inplace=True)  
  
merged2 = pd.merge(left=df1,  
                   right=df2_wide,  
                   left_on='Participant_id',  
                   right_on='Participant_id')
```

As both DataFrames will be joined on using the same column, you can simplify the command by using *on*

```
merged3 = pd.merge(left=df1,  
                   right=df2_wide,  
                   on='Participant_id')
```

df1, index is now '0'

Index	Participant_id	Session_id	_positive_thres	_positive_thre:
0	sub-1526180984	ses-V02	1.06759	1.07967

df2_wide, index is now '0'

Index	Participant_id	average_acc_Fi	verage_acc_Sa	_peak_acc_Fu:
0	sub-1526180984	0.802176	1.28725	1.40659

(not displayed)

Index	Participant_id	Session_id	_positive_thres	_positive_thre:
0	sub-1526180984	ses-V02	1.06759	1.07967

Make a large DataFrame containing all subjects

```
9 #%% Setup
10 import pandas as pd
11 import json
12
13 base_path = "/Volumes/HBCD/BIDS/hbcd/derivatives/hbcd_motion"
14 # Only including 3 cases for demonstration... there are 421 of them.
15 sub_id = ["sub-1526180984", "sub-3448939944", "sub-7333845703"]
16 sess_id = "ses-V02"
17 subfolder = "motion/Kinematics"
```

```
19 #%% Loading data and process
20 # We will save our manipulated dataframes in a list: dfs.
21 # Does it look any similar to assignment 3? Hope this helps you see
22 # why I prepared the assignment in such a way...
23 dfs = []
24 # Loop over `sub_id`:
25 # `sub` will be "sub-1526180984" in the first run,
26 # "sub-3448939944" the next run, and "sub-7333845703" in the last run
27 for sub in sub_id:
28     # JSON file name
29     filename = f"{sub}_{sess_id}_desc-kinematics_recording-25_motion.json"
30     # `join` using "/"
31     file_path = "/".join((base_path, sub, sess_id, subfolder, filename))
32
33     # `file_path` is the full path to the .json file
34     # How to open a .json file -> we saw this in week 1.
35     with open(file_path, 'rb') as f: Slide 4
36         data = json.load(f)
37
38     # Make a dataframe first
39     df1 = pd.DataFrame(data) Slide 5
40
41     # Use .reset_index() to get the nested dictionary's keys
42     # We now know what `inplace=True` does, right?
43     df1.reset_index(inplace=True) Slide 6
44
45     # Make a DataFrame with the following three columns
46     df2 = df1[['Participant_id', 'entropies', 'index']]
47     # Drop the two columns: 'entropies', 'index' Slide 7
48     df1.drop(columns=['entropies', 'index'], inplace=True)
49
50     # Make it a wide format: use `.pivot()` or `.pivot_table()`
51     df2_wide = df2.pivot(index='Participant_id',
52                           columns='index',
53                           values='entropies') Slides 8-10
54
55     # Set the index of df1
56     df1.set_index('Participant_id', inplace=True) Slides 12-13
57     df1.drop_duplicates(inplace=True)
58
59     # Merge the two DataFrames
60     merged = df1.merge(df2_wide, left_index=True, right_index=True)
61     Slides 14-16
62     # Add this merged DataFrame to `dfs`.
63     dfs.append(merged)
64
65     # Outside the loop, concatenate
66     all_subjects = pd.concat(dfs) This was introduced before too.
```

You will see this eventually

All three subjects' data are combined!

Participant_id	Session_id	_positive_thres	_positive_thre:	_negative_thres	_negative_thre	.eft_total_coun	ight_total_coui
sub-1526180984	ses-V02	1.06759	1.07967	-1.0537	-1.0058	22846	27869
sub-3448939944	ses-V02	1.12553	1.12846	-1.1191	-1.11094	27202	29664
sub-7333845703	ses-V02	1.11505	1.11814	-1.10578	-1.10648	49884	48230

To please (or disappoint) you...

- *pandas.json_normalize* can do a lot of benchmark for you!

If you don't add *ignore_index=True*, index values will be identical.

Index	Participant_id	Session_id	_positive_thres
0	sub-1526180984	ses-V02	1.06759
1	sub-3448939944	ses-V02	1.12553
2	sub-7333845703	ses-V02	1.11505

```
### Using json_normalize
from pandas import json_normalize

dfs = []
# Loop over `sub_id`:
# `sub` will be "sub-1526180984" in the first run,
# "sub-3448939944" the next run, and "sub-7333845703" in the last run
for sub in sub_id:
    # JSON file name
    filename = f"{sub}_{sess_id}_desc-kinematics_recording-25_motion.json"
    # `join` using "/"
    file_path = "/".join((base_path, sub, sess_id, subfolder, filename))

    # `file_path` is the full path to the .json file
    # How to open a .json file -> we saw this in week 1.
    with open(file_path, 'rb') as f:
        data = json.load(f)

    # Voila!
    normalized = json_normalize(data) SIMPLE!
    dfs.append(normalized)

all_subjects = pd.concat(dfs, ignore_index=True)
```

Make sure you
add this!

To please (or disappoint) you...

- Also, column names from the keys inside a nested dictionary will be combined, so you may want to rename them.

See how the first level key: 'entropies' and the second level key: 'Left_peak_acc_Sampen' are joined to make a new column name

[illegible]

But don't worry – there are plenty of scenarios where you will merge.

What's been processed until now is only about **File 1** of subjects.

You may want to merge data extracted from Files 2, 3, and 4 to this DataFrame.

Index	Participant_id	Session_id	_positive_thres
0	sub-1526180984	ses-V02	1.06759
1	sub-3448939944	ses-V02	1.12553
2	sub-7333845703	ses-V02	1.11505

Using .melt() to make long format

- Optional preprocessing for demo purpose

```
##% Make a long format

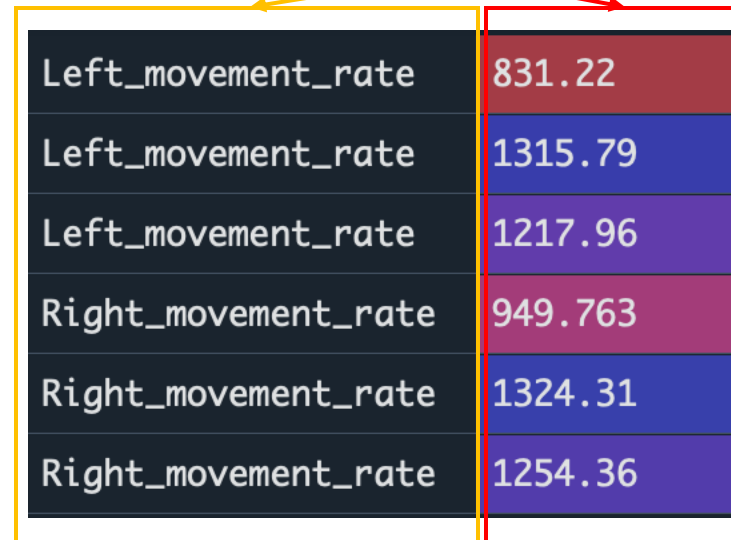
# First take a subset (optional)
variable_to_plot='movement_rate'
# We know that we want 'Left' and 'Right' <movement_rate> variables
subset = all_subjects[['Participant_id', f'Left_{variable_to_plot}', f'Right_{variable_to_plot}']]
```

subset

Index	Participant_id	Left_movement_rate	Right_movement_rate
0	sub-1526180984	831.22	949.763
1	sub-3448939944	1315.79	1324.31
2	sub-7333845703	1217.96	1254.36

Understanding long format

- We want one column of values and one column telling us what those values are.



Left_movement_rate	831.22
Left_movement_rate	1315.79
Left_movement_rate	1217.96
Right_movement_rate	949.763
Right_movement_rate	1324.31
Right_movement_rate	1254.36

- This is problematic, because rows are not “unique”. If you want to plot sub-1526180984’s **Left_movement_rate** value, how would you know?

Understanding long format

- That's why you also attach **unique identifiers**. Now you can tell each subject's variable value, correct?

sub-1526180984	Left_movement_rate	831.22
sub-3448939944	Left_movement_rate	1315.79
sub-7333845703	Left_movement_rate	1217.96
sub-1526180984	Right_movement_rate	949.763
sub-3448939944	Right_movement_rate	1324.31
sub-7333845703	Right_movement_rate	1254.36

Pandas by default
would call this a
'variable' column

Pandas by default
would call this a
'value' column

Understanding .melt()

```
# Use .melt()
melted = subset.melt(id_vars='Participant_id',
                    value_vars=[f'Left_{variable_to_plot}',
                                f'Right_{variable_to_plot}'])
```

value_vars

What columns of the original DataFrame(ex. subset) are going to be used to populate the 'value' column of a new DataFrame?

Index	Participant_id	Left_movement_rate	Right_movement_rate
0	sub-1526180984	831.22	949.763
1	sub-3448939944	1315.79	1324.31
2	sub-7333845703	1217.96	1254.36

Index	Participant_id	variable	value
0	sub-1526180984	Left_movement_rate	831.22
1	sub-3448939944	Left_movement_rate	1315.79
2	sub-7333845703	Left_movement_rate	1217.96
3	sub-1526180984	Right_movement_rate	949.763
4	sub-3448939944	Right_movement_rate	1324.31
5	sub-7333845703	Right_movement_rate	1254.36

id_vars

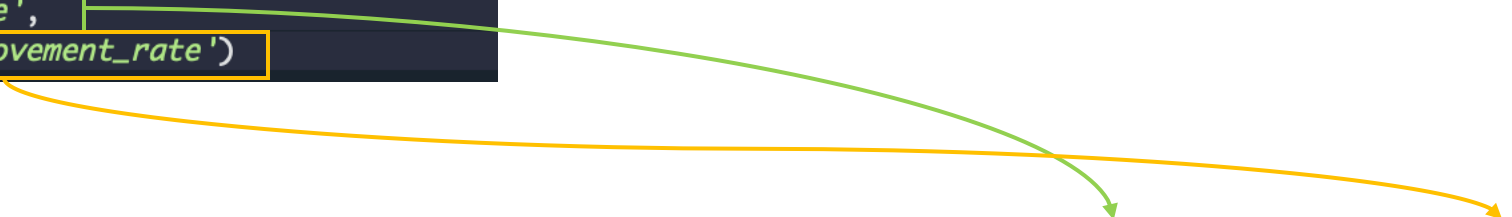
What columns of the original DataFrame(ex. subset) are going to be used as the unique identifier column of a new DataFrame?

Column names provided as the value of value_vars will be saved in the 'variable' column of a new DataFrame.

Understanding .melt()

```
# Use .melt()
melted = subset.melt(id_vars='Participant_id',
                    value_vars=[f'Left_{variable_to_plot}',
                                f'Right_{variable_to_plot}'],
                    var_name='side',
                    value_name='movement_rate')
```

Use **var_name** and **value_name** to customize the names of 'variable' and 'value' columns



Index	Participant_id	side	movement_rate
0	sub-1526180984	Left_movement_rate	831.22
1	sub-3448939944	Left_movement_rate	1315.79
2	sub-7333845703	Left_movement_rate	1217.96
3	sub-1526180984	Right_movement_rate	949.763
4	sub-3448939944	Right_movement_rate	1324.31
5	sub-7333845703	Right_movement_rate	1254.36